

OTEL-01: OpenTelemetry Fundamentals

Series: OTEL — OpenTelemetry Integration | **Notebook:** 1 of 8 | **Created:** January 2026 | **Last Updated:** 10/02/2026

Introduction to OpenTelemetry and Dynatrace

OpenTelemetry (OTel) is the industry-standard framework for collecting telemetry data. Dynatrace fully supports OpenTelemetry through native OTLP ingestion, allowing you to leverage OTel instrumentation while benefiting from Dynatrace's AI-powered analytics.

Table of Contents

- 1. [What is OpenTelemetry?](#)
- 2. [OTel vs. Proprietary Agents](#)
- 3. [Core Components](#)
- 4. [Signal Types: Traces, Metrics, Logs](#)
- 5. [Semantic Conventions](#)
- 6. [Dynatrace OTel Integration](#)
- 7. [When to Use OpenTelemetry](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with OTLP ingestion enabled
Permissions	Platform token: <code>openpipeline:traces:ingest</code> , <code>openpipeline:metrics:ingest</code> , <code>openpipeline:logs:ingest</code> . Classic access token: <code>openTelemetryTrace.ingest</code> , <code>metrics.ingest</code> , <code>logs.ingest</code>
Knowledge	Basic observability concepts

1. What is OpenTelemetry?

OpenTelemetry is a vendor-neutral, open-source observability framework that provides:

Component	Purpose
APIs	Standardized interfaces for instrumentation
SDKs	Language-specific implementations
Collector	Agent for receiving, processing, and exporting telemetry
Protocol (OTLP)	Standard wire protocol for telemetry data

CNCF Project

OpenTelemetry is a Cloud Native Computing Foundation (CNCF) project, formed from the merger of OpenTracing and OpenCensus.

Key Benefits

Benefit	Description
Vendor Neutral	No lock-in to specific backends
Standardized	Consistent across languages and platforms
Comprehensive	Traces, metrics, and logs in one framework
Community-Driven	Active development, broad adoption

2. OTel vs. Proprietary Agents

Comparison

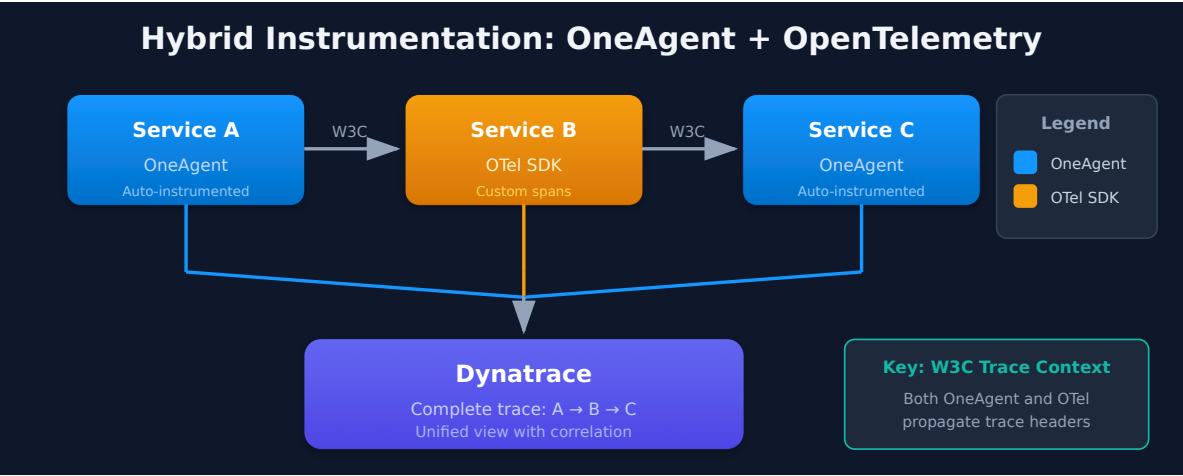
Aspect	OpenTelemetry	Dynatrace OneAgent
Instrumentation	Manual or auto-instrumentation	Automatic
Deployment	SDK + Collector	Single agent
Coverage	Code-level	Full-stack
Flexibility	High (any backend)	Dynatrace-specific
Maintenance	Higher	Lower
Features	Standard signals	AI, RCA, Dynatrace Intelligence

When to Choose OTel

Scenario	Recommendation
Multi-vendor observability	OTel
Serverless/Lambda	OTel
Custom instrumentation needs	OTel
Existing OTel investment	OTel
Full-stack visibility	OneAgent + OTel
Minimal effort	OneAgent

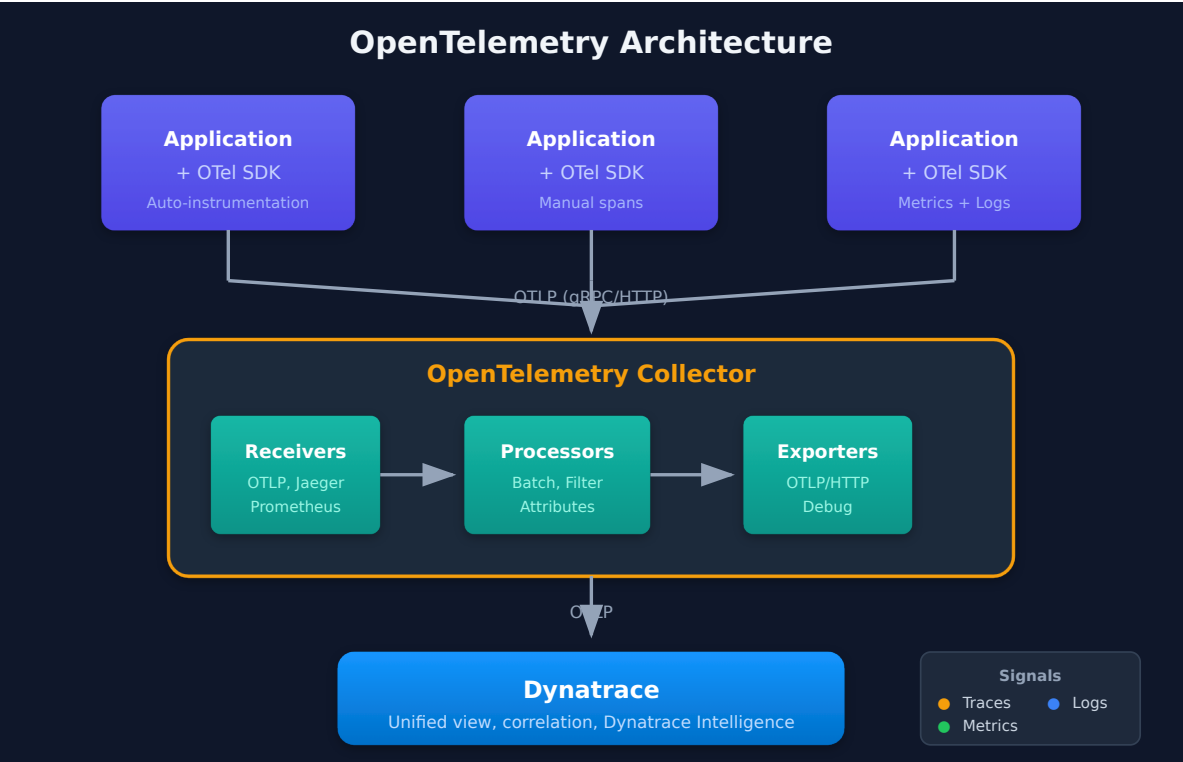
Hybrid Approach

Dynatrace supports using both OneAgent and OTel together:



3. Core Components

OpenTelemetry Architecture



Component Descriptions

Component	Role
API	Defines how to instrument (language-specific interfaces)
SDK	Implements the API, handles sampling, batching
Exporter	Sends data to backends (OTLP, Jaeger, etc.)
Collector	Standalone service for processing telemetry
Instrumentation Libraries	Pre-built instrumentation for frameworks

4. Signal Types: Traces, Metrics, Logs

Traces

Distributed traces track requests across services.

Concept	Description
Trace	End-to-end transaction path
Span	Single operation within a trace
SpanContext	Propagated context (trace ID, span ID)
Attributes	Key-value metadata on spans
Events	Timestamped annotations on spans

Metrics

Quantitative measurements over time.

Instrument Type	Use Case	Example
Counter	Monotonically increasing value	Request count
UpDownCounter	Value that goes up and down	Active connections
Histogram	Distribution of values	Response time
Gauge	Current value	Temperature, queue size

Logs

Structured event records correlated with traces.

Field	Description
Timestamp	When the event occurred
Severity	Log level (DEBUG, INFO, ERROR)
Body	Log message content
Attributes	Structured metadata
TraceContext	Link to active span

5. Semantic Conventions

Semantic conventions define standard attribute names for consistent telemetry.

Note: OpenTelemetry semantic conventions are evolving. HTTP conventions were **stabilized in 2023** with new names (see migration tables below). Database conventions are also migrating. SDKs support a gradual transition via the `OTEL_SEMCONV_STABILITY_OPT_IN` environment variable. Both old and new names may coexist during migration.

Resource Attributes

Attribute	Example	Description
<code>service.name</code>	<code>checkout-api</code>	Logical service name
<code>service.version</code>	<code>1.2.3</code>	Service version
<code>service.namespace</code>	<code>shop</code>	Groups services (team, system or application); scopes <code>service.name</code>
<code>deployment.environment.name</code>	<code>prod</code>	Deployment environment

Note: `deployment.environment` was renamed to `deployment.environment.name` in the stable resource semantic conventions.

HTTP Span Attributes

Old (Legacy)	Stable	Description
<code>http.method</code>	<code>http.request.method</code>	HTTP method
<code>http.url</code>	<code>url.full</code>	Full URL
<code>http.status_code</code>	<code>http.response.status_code</code>	Response status code
<code>http.route</code>	<code>http.route</code> (unchanged)	Route template
<code>http.target</code>	<code>url.path</code> + <code>url.query</code>	Request target
<code>http.host</code>	<code>server.address</code>	Server hostname

Tip: Use `OTEL_SEMCONV_STABILITY_OPT_IN=http` to emit only stable names, or `http/dup` to emit both old and new during migration. See the [HTTP semconv migration guide](#).

Database Span Attributes

Old (Legacy)	Stable	Description
<code>db.name</code>	<code>db.namespace</code>	Database name/namespace
<code>db.statement</code>	<code>db.query.text</code>	Query statement
<code>db.operation</code>	<code>db.operation.name</code>	Operation type
<code>db.system</code>	<code>db.system</code>	Database type (unchanged)

Tip: Use `OTEL_SEMCONV_STABILITY_OPT_IN=database` for stable database names. See the [DB semconv migration guide](#).

Kubernetes Attributes

Attribute	Example	Description
<code>k8s.namespace.name</code>	<code>checkout</code>	Namespace
<code>k8s.pod.name</code>	<code>checkout-api-abc123</code>	Pod name
<code>k8s.deployment.name</code>	<code>checkout-api</code>	Deployment

6. Dynatrace OTel Integration

OTLP Endpoints

Dynatrace accepts OTLP data natively via **HTTP only**, with binary Protocol Buffers: *"gRPC is not supported. API calls need to use HTTP."* and *"JSON is not supported for Protocol Buffers. Binary format must be used."*

Protocol	Endpoint	Notes
HTTP	<code>https://{your-env}.live.dynatrace.com/api/v2/otlp</code>	Recommended for direct ingest
gRPC	Not supported for direct Dynatrace ingest	Use a Collector to convert gRPC → HTTP

Important: If your application uses gRPC to export OTLP data, route it through an OpenTelemetry Collector configured with an `otlp gRPC` receiver and an `otlphttp` exporter to Dynatrace. See [Transform OTLP gRPC](#).

Authentication

Two token types work, with different schemes and scopes:

Token	Header	Traces	Metrics	Logs
Platform token	Authorization: Bearer <token>	<code>openpipeline:traces:ingest</code>	<code>openpipeline:metrics:ingest</code>	<code>openpipeline:logs:ingest</code>
Classic access token	Authorization: Api-Token <token>	<code>openTelemetryTrace.ingest</code>	<code>metrics.ingest</code>	<code>logs.ingest</code>

Sources: [OTLP API \(DT docs\)](#) — "Platform token: Use Bearer in the Authorization header. The required scopes are `openpipeline:logs:ingest`, `openpipeline:metrics:ingest`, and `openpipeline:traces:ingest`."

Configuration Example

OTel Collector Exporter:

```
exporters:
  otlphttp:
    endpoint: https://{your-env}.live.dynatrace.com/api/v2/otlp
    headers:
      Authorization: Api-Token ${DT_API_TOKEN}
```

SDK Direct Export (Python):

```
import os

from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter

exporter = OTLPSpanExporter(
    endpoint="https://{your-env}.live.dynatrace.com/api/v2/otlp/v1/traces",
    headers={"Authorization": f"Api-Token {os.environ['DT_API_TOKEN']}"})
```

OpenTelemetry Licensing & Cost

OpenTelemetry data is billed exactly like any other ingested data under the Dynatrace Platform Subscription (DPS) — there is no separate OTel licence and no OTel surcharge or discount. Each signal maps to DPS capabilities:

Signal	Ingest & Process	Retain	Query
Traces	per GiB ingested	GiB-days (10 days included)	per GiB scanned (Distributed Tracing / Services apps don't bill queries)
Metrics	per 100k data points	GiB-days (15 months @ 1-min granularity included)	included with Ingest & Process
Logs	per GiB ingested	GiB-days	per GiB scanned

Included volume follows the source, not the protocol: Full-Stack Monitoring includes a defined trace volume *"for traces that are sent via the OneAgent Trace API or from a Full-Stack monitored host"*, and *"Metrics originating from Full-Stack-monitored hosts or containers include a defined amount of metric data points"* — you are charged only for data above the included amount. OTel telemetry from sources outside Full-Stack Monitoring (for example, a workload on a host with no OneAgent) has no included volume to draw on and bills on the rate card as ingested. Practical consequence: high-volume OTel traces/metrics from such sources are a direct cost line — size and (if needed) sample them deliberately (see OTEL-04 §7 and the FINOPS series).

Sources: [OpenTelemetry licensing \(DT docs\)](#) — *"With Dynatrace, OpenTelemetry data is treated like any other ingested data."* **Derived:** OTel data from sources outside Full-Stack Monitoring has no included volume, because the page scopes the allowance to trace and metric data from those sources.

```
// View OpenTelemetry traces that arrived over OTLP
// dt.openpipeline.source identifies the ingest path. otel.scope.name is NOT a reliable
// "is OTel" test: OneAgent spans can carry an instrumentation scope too.
fetch spans, from:-1h
| filter dt.openpipeline.source == "/api/v2/otlp/v1/traces"
| fields start_time, trace.id, service.name, span.name, otel.scope.name, duration
| sort start_time desc
| limit 20
```

```
// OTel instrumentation libraries in use (OTLP-ingested spans)
fetch spans, from:-1h
| filter dt.openpipeline.source == "/api/v2/otlp/v1/traces"
| summarize count = count(), by:{otel.scope.name, otel.scope.version}
| sort count desc
| limit 20
```

7. When to Use OpenTelemetry

OTel is Ideal For

Scenario	Why OTel
Serverless	AWS Lambda, Azure Functions (no agent)
Multi-cloud	Consistent instrumentation across clouds
Custom metrics	Business-specific measurements
Vendor diversity	Send to multiple backends
Edge/IoT	Lightweight collection

Scenario	Why OTel
Polyglot	Many languages, one standard

OneAgent is Better For

Scenario	Why OneAgent
Full-stack	Infrastructure + code in one
Zero-effort	Auto-instrumentation
Dynatrace Intelligence	Full AI capabilities
PurePath	Complete transaction tracing
Real User Monitoring	RUM integration

Recommended: Hybrid

- For most enterprises:
- **OneAgent** for infrastructure and supported technologies
 - **OTel** for unsupported languages, serverless, custom metrics

Next Steps

Now that you understand OTel fundamentals, proceed to:

Next Notebook	Topic
OTEL-02: Collector Architecture	Deep-dive into the Collector
OTEL-03: Collector Deployment	Deployment patterns
OTEL-04: Trace Instrumentation	Instrumenting your code

Summary

In this notebook, you learned:

- What OpenTelemetry is and its benefits
- How OTel compares to proprietary agents
- Core OTel components: API, SDK, Collector
- Signal types: traces, metrics, logs
- Semantic conventions for consistent telemetry (including the stable HTTP and DB naming migration)
- Dynatrace OTLP integration configuration (HTTP only — gRPC requires Collector)
- When to use OTel vs. OneAgent

References

- [OpenTelemetry Official Docs](#)
- [Dynatrace OpenTelemetry Integration](#)

- [OTLP Specification](#)
- [Semantic Conventions](#)
- [HTTP Semconv Migration Guide](#)
- [DB Semconv Migration Guide](#)

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace.
Always verify information against official Dynatrace documentation.*